

FE522 Final Project

You Wang

ywang408@stevens.edu

Stevens Institute of Technology

Spring 2026

Overview

Logistics

- ▶ **Presentation:** May 7 (in-class, ~10 minutes per group)
- ▶ **Code submission:** May 10
- ▶ **Weight:** 30% of final grade
- ▶ **Submission:** Via canvas
- ▶ **Team size:** 1–3 people

Goal:

- ▶ Design and build a C++ program related to computational finance or numerical computing.
- ▶ Full creative freedom over topic and program design.

Minimum requirement

- ▶ **At least as complex as Assignment 2** for a solo project.
 - Assignment 2 had three interacting components, stateful lambdas, STL algorithms, and a CMake build.
- ▶ Larger teams are expected to deliver proportionally more complexity.
- ▶ A single class with a few member functions is too simple.
- ▶ We expect a system with multiple components that work together.

Your project must include:

- ▶ **Multiple source files** with a reasonable layout (not everything in one file).

Minimum requirement

- ▶ A working **CMakeLists.txt or Makefile** that builds the project.
- ▶ A **README.md** (see Deliverables).

Topic Selection

Free choice

- ▶ You are free to select any topic related to computational finance or numerical computing.
- ▶ Below are two suggested topics with detailed specifications.
 - You may also choose from additional examples at the end.

Topic A: Implied Volatility Surface Builder

- ▶ Read a CSV of option market data.
- ▶ Compute Black-Scholes implied volatility for each contract.
- ▶ Display the results as a volatility surface.

Required functionality:

- ▶ Parse option data from CSV.
 - Each row: underlying price, strike, expiry, option type, risk-free rate, market price.
- ▶ Implement Black-Scholes pricing for European calls and puts.
- ▶ Use a root-finding algorithm (bisection or Newton's method) to solve for implied volatility.

Topic A (cont.)

- ▶ Organize results into a surface indexed by strike and expiry.
- ▶ Print a formatted table of the surface.
- ▶ Compute summary statistics:
 - ATM implied volatility per expiry.
 - Skew: OTM put IV minus ATM IV.
 - Flag contracts where the solver failed to converge.

Topic B: Yield Curve Bootstrapper

- ▶ Read a CSV of par bond data.
- ▶ Bootstrap a zero-coupon yield curve.
- ▶ Price other bonds and compute risk measures.

Required functionality:

- ▶ Parse par bond data from CSV.
 - Each row: maturity, coupon rate, face value, payment frequency.
- ▶ Bootstrap the zero-coupon yield curve:
 - For each maturity, solve for the spot rate that prices the bond at par.

Topic B: Yield Curve Bootstrapper

- Use previously bootstrapped rates for earlier cash flows.
- Use a root-finding algorithm where a closed-form solution is not available.

Topic B (cont.)

- ▶ Support interpolation between bootstrapped maturities.
- ▶ Linear interpolation on rates is sufficient.
- ▶ Price non-par bonds using the bootstrapped curve:
 - Discount each cash flow at the interpolated zero rate.
 - Sum to get present value.
- ▶ Compute duration and convexity for each priced bond.
- ▶ Print formatted output:
 - The bootstrapped zero curve.
 - Priced bonds with PV, duration, convexity.
 - Diagnostics (e.g., which maturities required numerical solving).

Deliverables

What to submit

1. **Source code:**

- ▶ Clean, compiling C++ code.
- ▶ Must compile with `g++` or `clang++` using `-std=c++17`, no errors.

2. **README.md** (required):

- ▶ What the program does.
- ▶ How to build and run it.
- ▶ Any third-party dependencies.

3. **Project report** (1–2 pages, Markdown):

- ▶ Architecture overview: how components fit together.

What to submit

- ▶ Mathematical methods: state formulas, explain algorithms, cite sources.
- ▶ Key design decisions and trade-offs.
- ▶ What you would improve given more time.

Grading

Policy

- ▶ You **may** use LLMs (ChatGPT, Copilot, Claude, etc.) as a development tool.
- ▶ During the **presentation**, you will be asked detailed questions about your code.
 - You must be able to explain every part of your submission.
 - Inability to answer will be reflected in the presentation grade.

Rubrics

Presentation:

- ▶ Present your system design and demonstrate the program.
- ▶ Respond to questions on design decisions, edge cases, and memory management.

Code:

- ▶ Evaluated on correctness, architecture, and design quality.
- ▶ Class structure and use of language features matter.

Rubrics

Presentation:

- ▶ **README:** a reader must be able to build and run the project from it alone.
- ▶ **Project report:** clear explanation of the mathematics and thoughtful reflection on trade-offs.